

Innleverings-PDF

IND320 -- Ass 2

General

Work Log (Part 1)

In this first part of the project, I built a basic analytics workflow using both Jupyter Notebook and Streamlit, aiming for a simple but clear data dashboard.

Jupyter Notebook. I loaded the supplied CSV with Pandas, explored the schema, and displayed the contents in a tidy, readable format. I produced exploratory plots for individual columns as well as a combined plot to compare trends across all columns. All cells were executed so that outputs and figures are visible in the exported PDF. Throughout the notebook I added explanatory comments to make the code easy to follow and reproducible if someone inherits the project.

Streamlit app. I structured the app into four pages: a Home page, a Data Table page, a Plot page, and an About page. The Home page includes sidebar navigation to the other pages. On the Data Table page, the CSV is rendered along with a small line chart per column for the first month in the dataset. On the Plot page, users can choose one column—or all columns—from a dropdown and filter which months to show via a slider. I enabled caching so the CSV loads faster, and I checked the app both locally and on Streamlit Cloud. The CSV is included in the repository so the app runs in both environments.

Challenges and fixes. Early on I struggled with file formats and repository layout. Streamlit does not run `.ipynb` files directly, so I split relevant logic into `.py` modules while keeping the `.ipynb` for documentation and exploration. I also ran into issues after reorganizing folders—Streamlit Cloud and GitHub didn't always find files. I resolved this by keeping `streamlit_app.py` and `utils.py` in the repository root and by switching to relative paths for loading the CSV. Finally, I cleaned up `requirements.txt` to include only necessary packages, which stabilized the cloud deployment.

Use of AI. I used AI tools (e.g., ChatGPT) to troubleshoot specific issues: diagnosing missing files and bad imports, setting up Streamlit caching, improving the repository structure, and understanding Streamlit Cloud error logs. AI also helped me reason about minimal dependencies for `requirements.txt`. I did **not** use AI to auto-generate full analysis code or the application itself. The core data loading, plotting logic, page structure, and final code comments were written by me. Decisions about the UI/UX (page layout, controls, and which charts to include) were also my own.

Links.

GitHub repository: https://github.com/Sheikh20o3/IND320_1_Project

Streamlit app: <https://streamv2.streamlit.app/>

Read the supplied CSV file and Imports

```
In [179.. import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import subprocess

# Load the dataset
df = pd.read_csv("/Users/a.h.sheikh/Desktop/IND320_Git_Job/IND320_1_Proje
df['time'] = pd.to_datetime(df['time'], errors='coerce')
```

Visualisation

```
In [180.. # Load the dataset
df = pd.read_csv("/Users/a.h.sheikh/Desktop/IND320_Git_Job/IND320_1_Proje
df['time'] = pd.to_datetime(df['time'], errors='coerce') # datetime form
df['month'] = df['time'].dt.month # create a month column for grouping

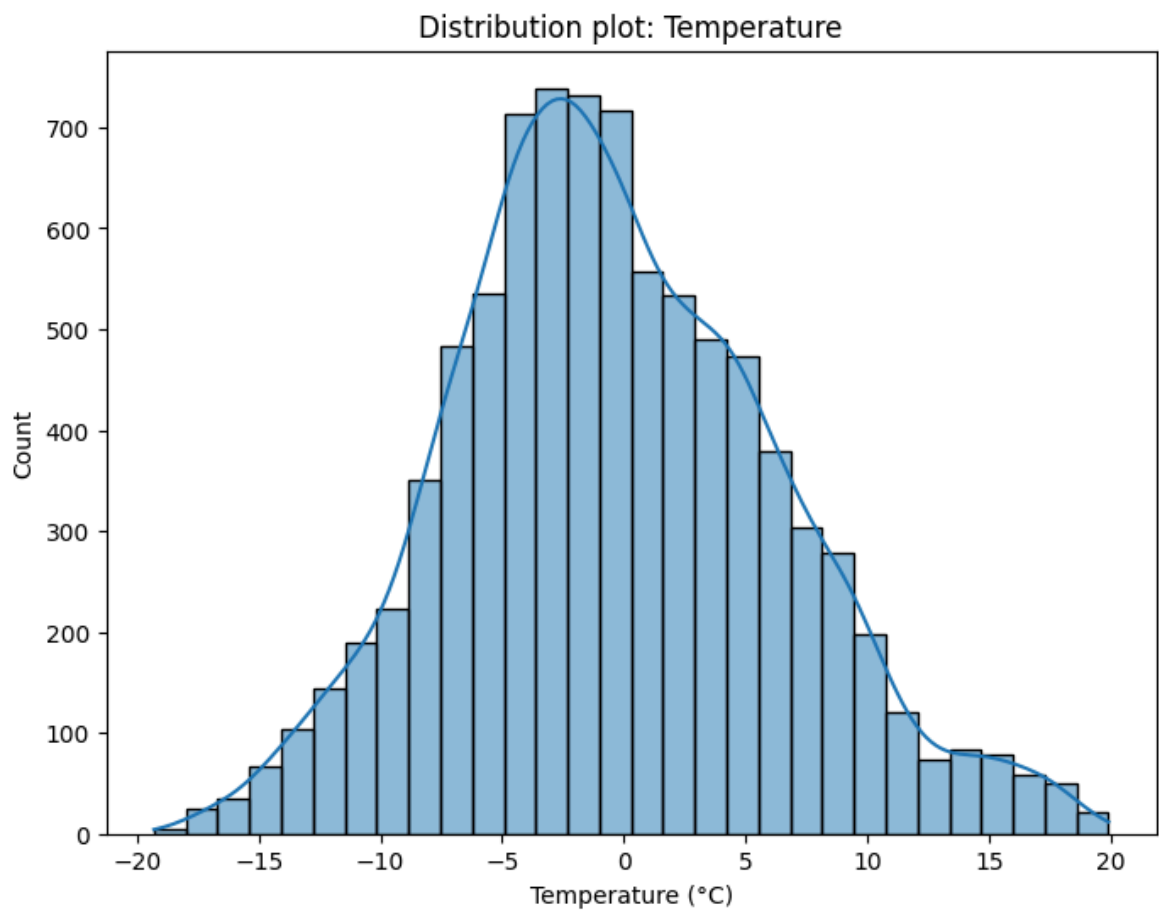
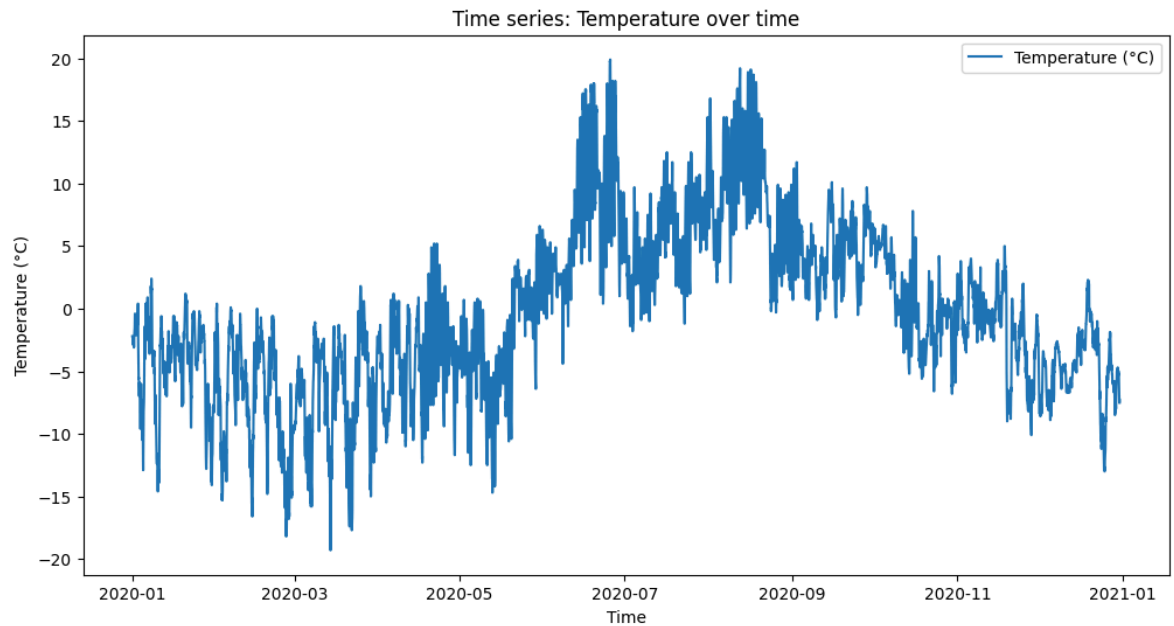
# 1. Time series plot
plt.figure(figsize=(12,6))
plt.plot(df['time'], df['temperature_2m (°C)'], label="Temperature (°C)")
plt.xlabel("Time")
plt.ylabel("Temperature (°C)")
plt.title("Time series: Temperature over time")
plt.legend()
plt.show()

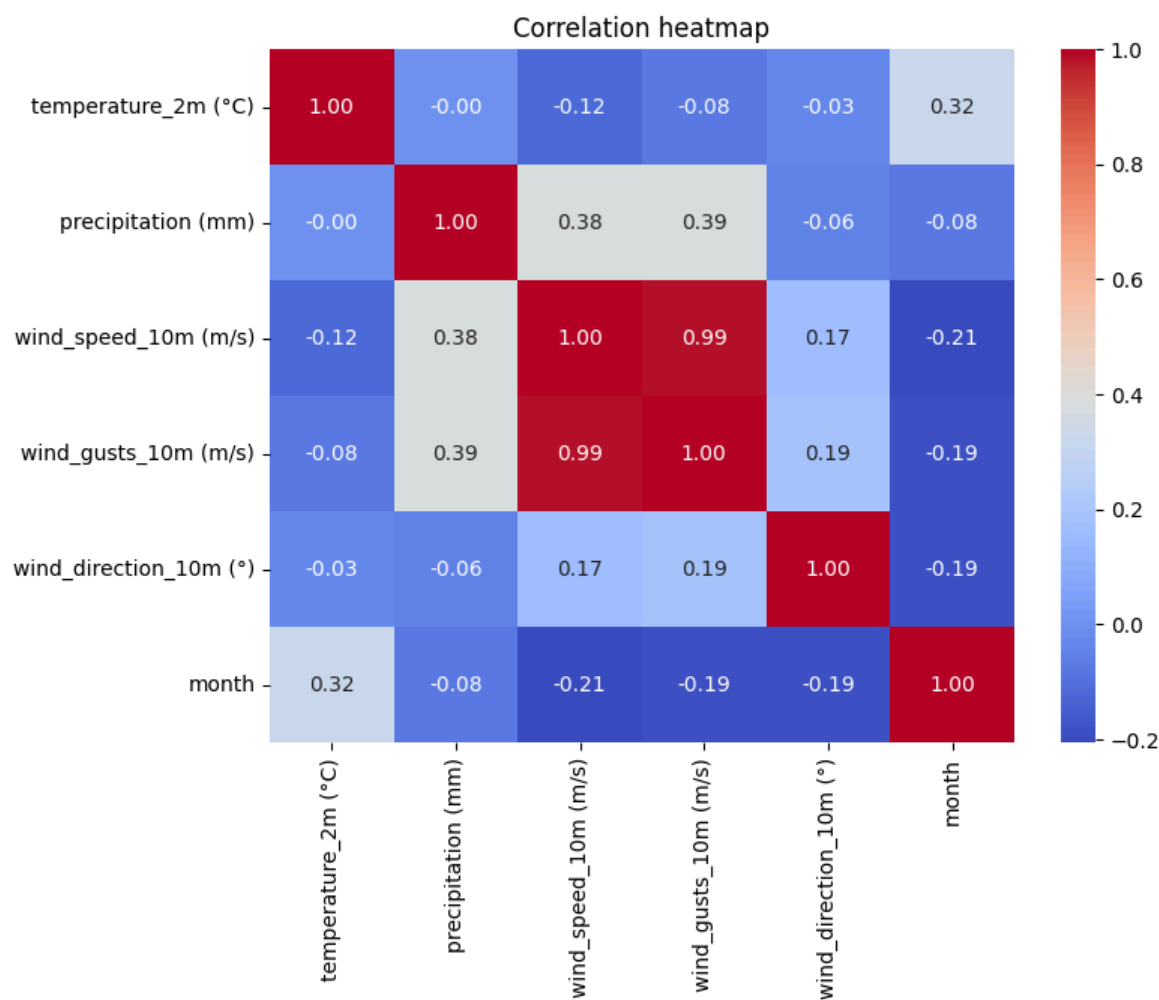
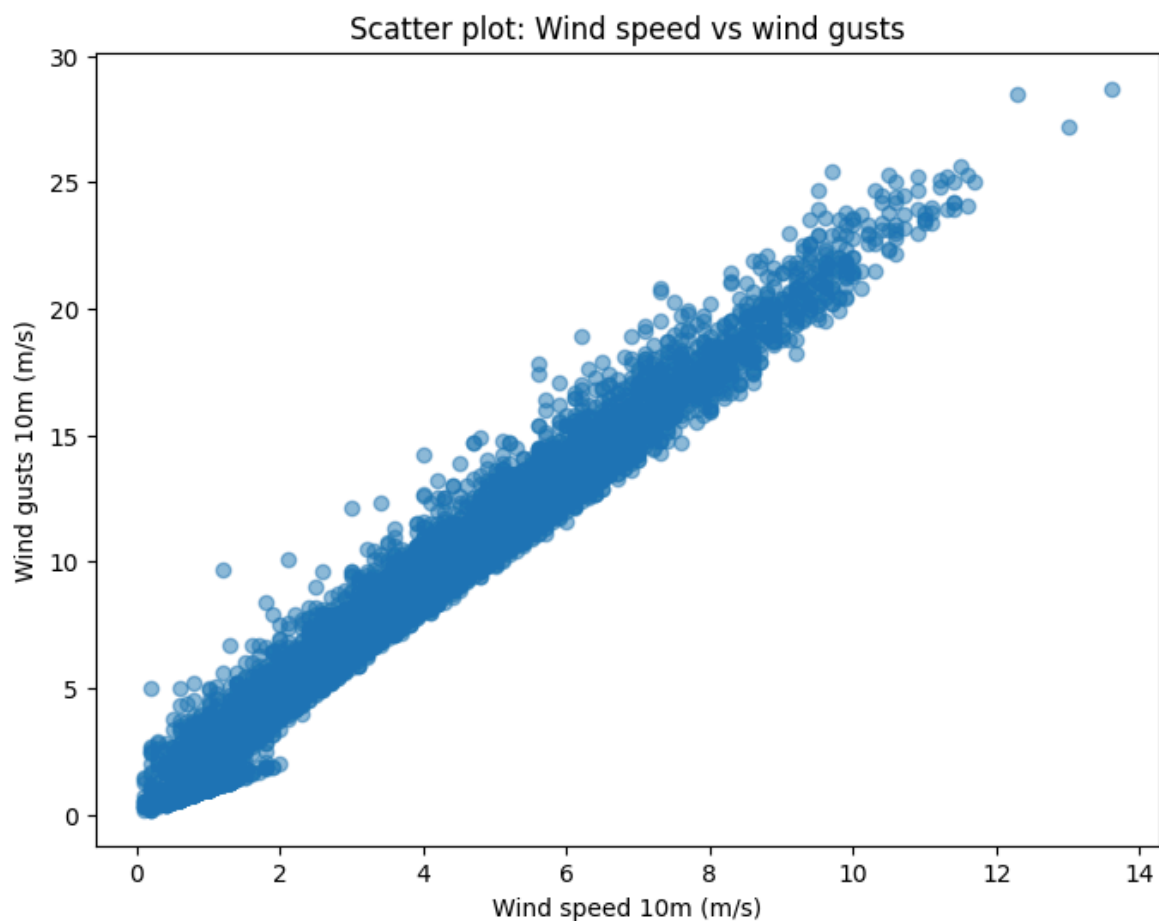
# Histogram (distribution of temperature)
plt.figure(figsize=(8,6))
sns.histplot(df['temperature_2m (°C)'], bins=30, kde=True)
plt.xlabel("Temperature (°C)")
plt.title("Distribution plot: Temperature")
plt.show()

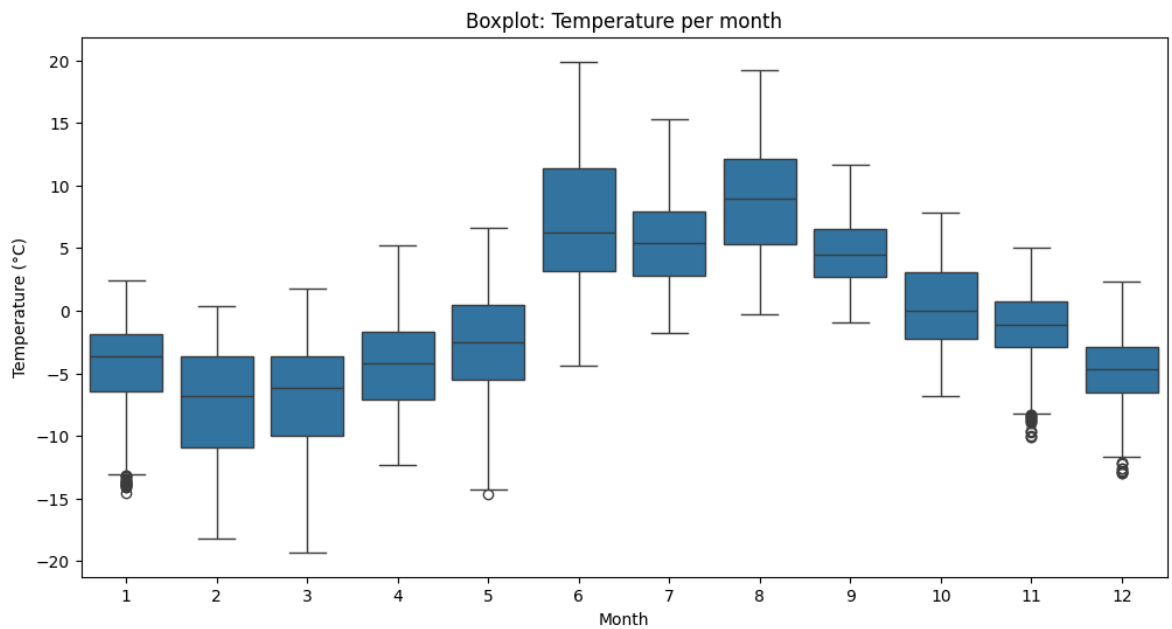
# Scatter plot
plt.figure(figsize=(8,6))
plt.scatter(df['wind_speed_10m (m/s)'], df['wind_gusts_10m (m/s)'], alpha
plt.xlabel("Wind speed 10m (m/s)")
plt.ylabel("Wind gusts 10m (m/s)")
plt.title("Scatter plot: Wind speed vs wind gusts")
plt.show()

# Correlation heatmap
plt.figure(figsize=(8,6))
corr = df.select_dtypes(include="number").corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation heatmap")
plt.show()
```

```
# Boxplot of temperature per month
plt.figure(figsize=(12,6))
sns.boxplot(x='month', y='temperature_2m (°C)', data=df)
plt.xlabel("Month")
plt.ylabel("Temperature (°C)")
plt.title("Boxplot: Temperature per month")
plt.show()
```

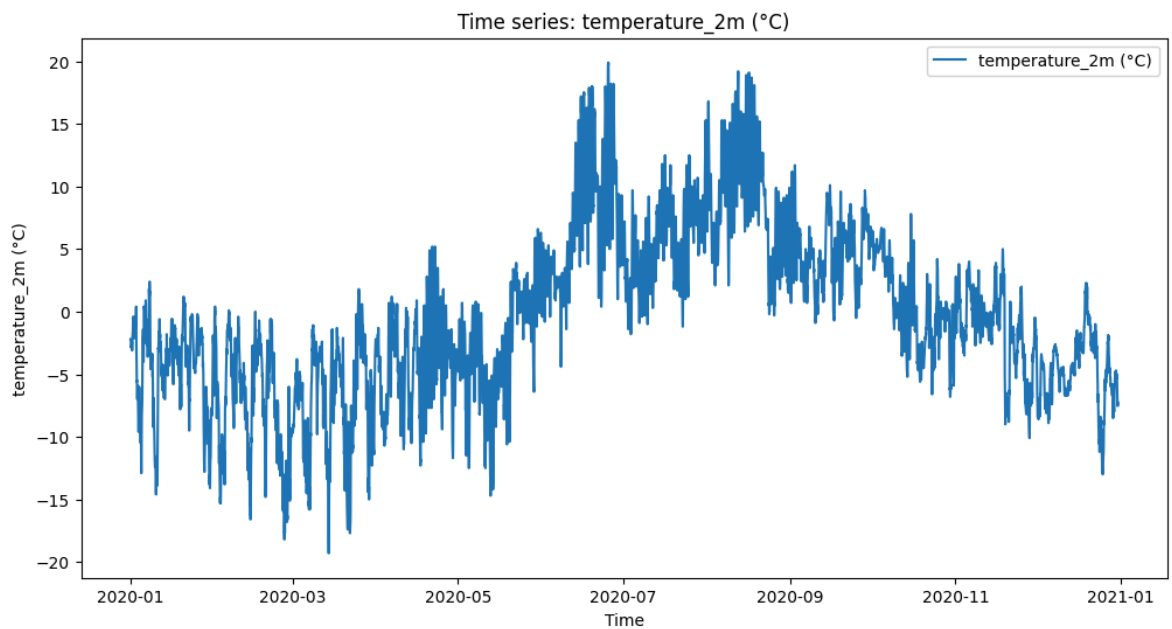


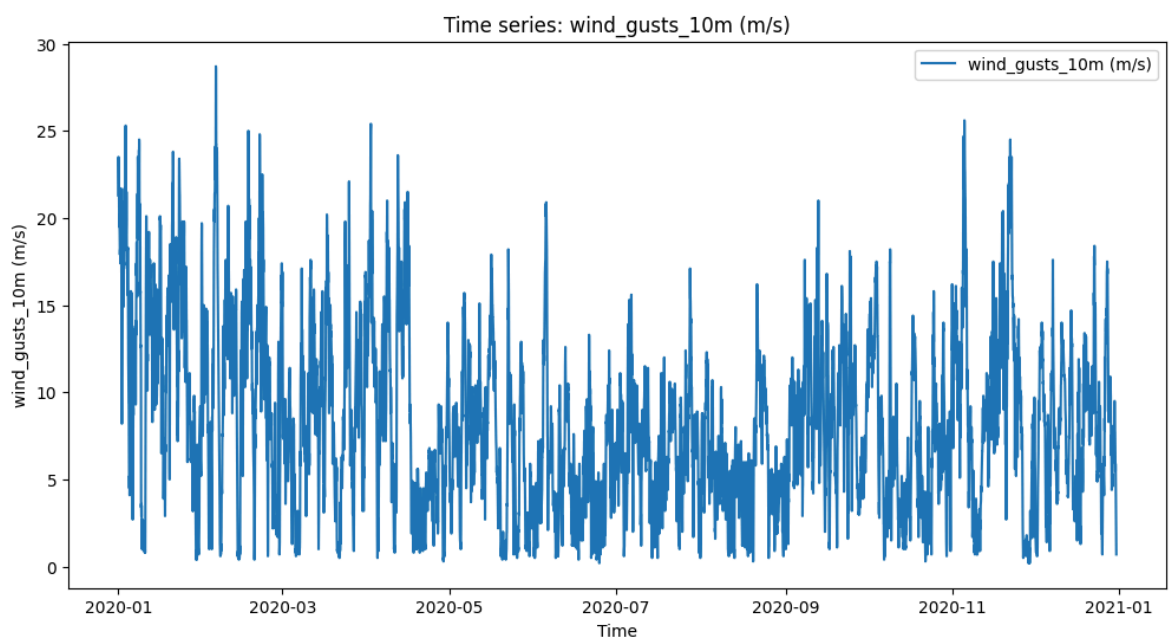
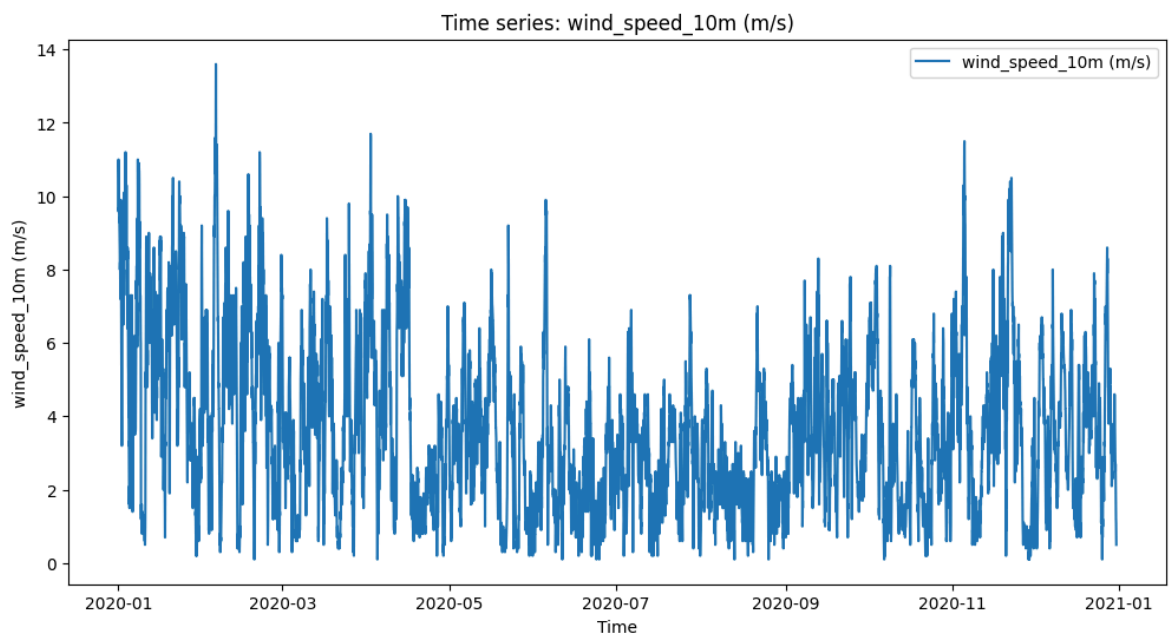
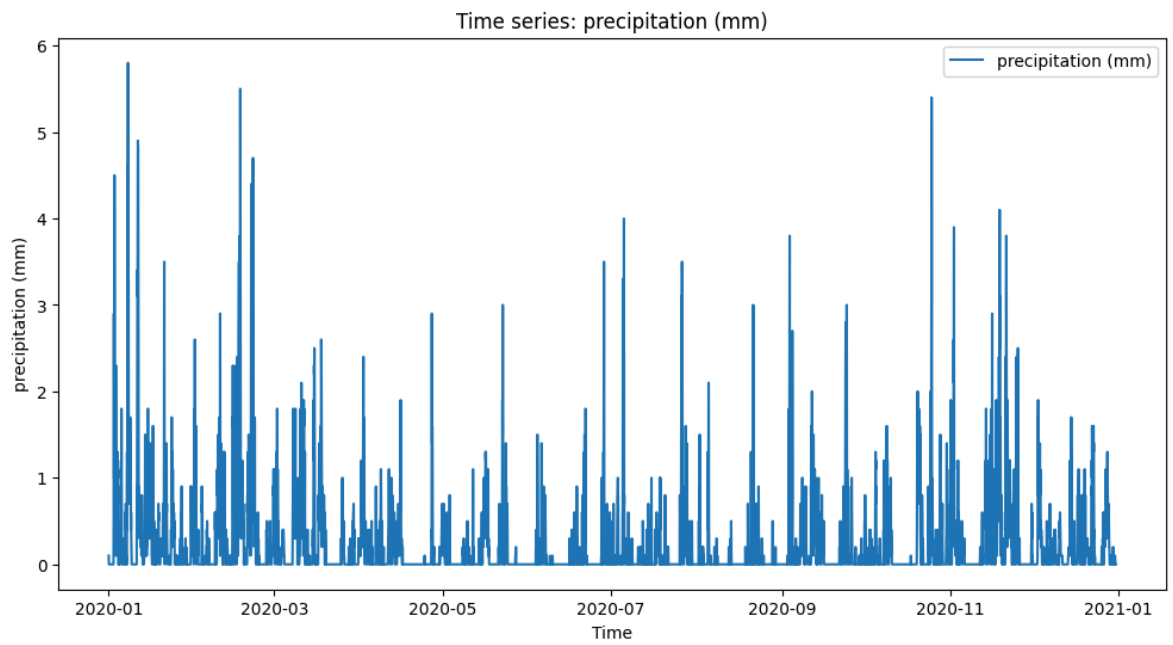


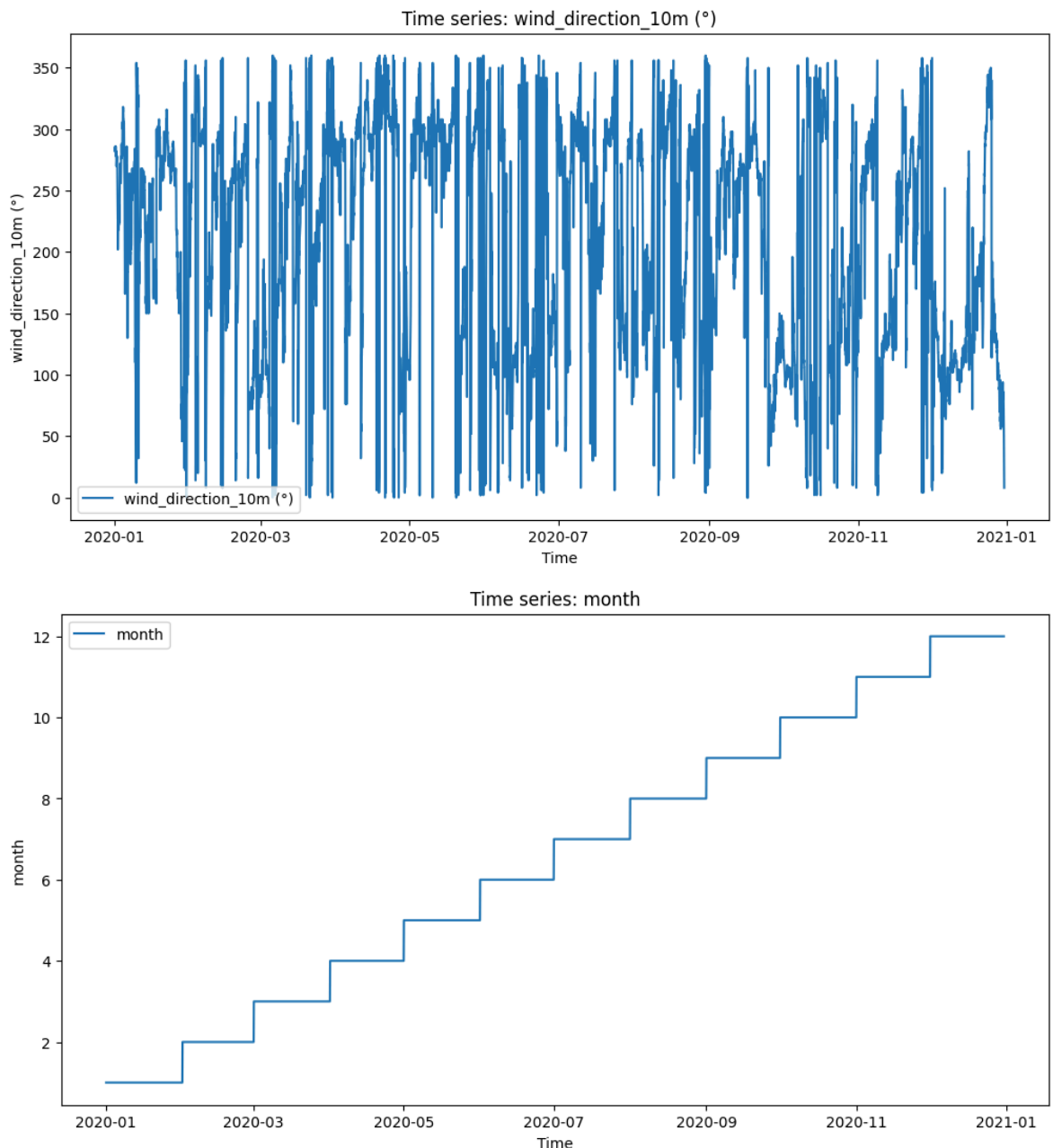


Plot each column separately.

```
In [181]: # Loop through all numeric columns and plot them separately
for col in df.select_dtypes(include="number").columns:
    plt.figure(figsize=(12,6))
    plt.plot(df['time'], df[col], label=col)
    plt.xlabel("Time")
    plt.ylabel(col)
    plt.title(f"Time series: {col}")
    plt.legend()
    plt.show()
```







Reflection

When plotting each variable individually it quickly becomes clear that the scales are very different. Temperature ranges between about -20 and $+20$ degrees, precipitation is mostly small spikes that do not exceed a few millimeters, wind speed and gusts vary within a completely different range, and wind direction covers the full circle of 0 to 360 degrees. If these series were shown on the same axis the smaller values would be overshadowed and almost invisible. With each variable having its own figure it becomes natural to read the changes and patterns over time because every plot uses a scale that fits the data it shows. This way we can see the seasonal cycle of temperature, the irregular rainfall events, and the dynamics of wind without the distortions that would occur if everything was forced into a single graph.

Plot all columns together

Doing it by using multiple y-axes

```
In [182... import matplotlib.pyplot as plt

time_col = "time"
value_cols = [c for c in df.columns if c != time_col]

fig, ax0 = plt.subplots(figsize=(14, 7))

axes = [ax0]
lines = []

for i, col in enumerate(value_cols):
    if i == 0:
        ax = ax0
    else:
        ax = ax0.twinx()
        # flytt hver ekstra høyre-akse litt ut til høyre
        offset = 1 + 0.1 * (i - 1)
        ax.spines.right.set_position(("axes", offset))
        axes.append(ax)

    ln, = ax.plot(df[time_col], df[col], label=col)
    ax.set_ylabel(col, color=ln.get_color())
    ax.tick_params(axis='y', labelcolor=ln.get_color())
    lines.append(ln)

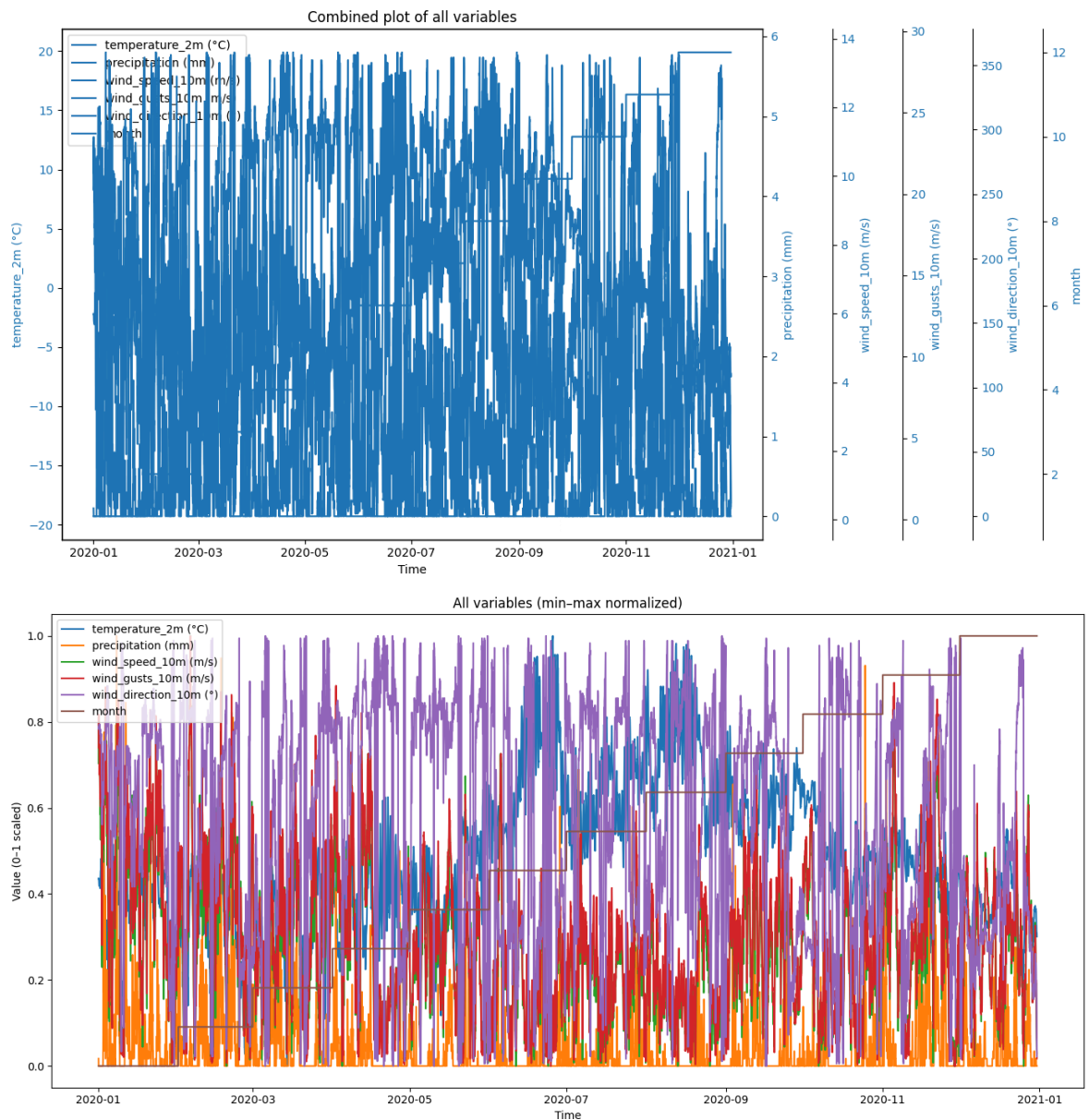
ax0.set_xlabel("Time")
ax0.set_title("Combined plot of all variables")
fig.tight_layout()

# felles legend med alle serier
ax0.legend(lines, [l.get_label() for l in lines], loc="upper left", bbox_
plt.show()

norm = (df[value_cols] - df[value_cols].min()) / (df[value_cols].max() -

plt.figure(figsize=(14, 7))
for col in value_cols:
    plt.plot(df[time_col], norm[col], label=col)

plt.xlabel("Time")
plt.ylabel("Value (0-1 scaled)")
plt.title("All variables (min-max normalized)")
plt.legend(loc="upper left")
plt.tight_layout()
plt.show()
```

Reflection

When plotting all columns together is the challenge that the variables are measured on very different scales. Temperature is expressed in Celsius and ranges between – 20 and +20. Where the precipitation is measured in millimeters and often not exceeds 6, The wind speed is measured in meters per second and reaches 14. The wind direction is in degrees from 0 to 360. I plotted all columns together using multiple y-axes. This approach preserves the original units of each variable and makes the figure easier to interpret. Here can I read the values of temperature, precipitation and wind speed. If all series were forced onto a single scale, the small variations in precipitation would disappear compared to the much larger ranges of wind direction or temperature. When each variable have its own axis, the differences become visible while still being displayed in the same figure. This makes the visualization easier to read, since it reflects the true magnitudes of the data rather than abstract normalized values.

Assignment 2 (Part 2 for the API, Cassandra, Spark og MongoDB)

```
In [183... import os, sys

PROJECT_ROOT = r"/Users/a.h.sheikh/Desktop/IND320_Git_Job/IND320_1_Projec
if os.path.isdir(PROJECT_ROOT):
    os.chdir(PROJECT_ROOT)
    print("Arbeidskatalog satt til:", os.getcwd())
else:
    print("Fant ikke prosjektrot:", PROJECT_ROOT, "-- kjører i nåværende

if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

for p in ["utils_elhub.py", "utils.py", "load_to_mongo.py", "streamlit_ap
print("Finnes" if os.path.exists(p) else "Mangler", "→", p)
```

```
Arbeidskatalog satt til: /Users/a.h.sheikh/Desktop/IND320_Git_Job/IND320_1
_Project
Finnes → utils_elhub.py
Finnes → utils.py
Finnes → load_to_mongo.py
Finnes → streamlit_app.py
Finnes → pages/4_Elhub_Production.py
```

```
In [184... # OVERRIDE: Sett MongoDB Atlas-URI eksplisitt for kjøring
MONGO_URI = "mongodb+srv://AHS_db_user:fAdp5LC8Gg1DRedl@ahs786student.qh8
MONGO_DB = "elhub"
MONGO_COLL = "production_2021_by_hour"
print("MONGO_URI er satt. DB:", MONGO_DB, "Collection:", MONGO_COLL)
```

```
MONGO_URI er satt. DB: elhub Collection: production_2021_by_hour
```

```
In [185... from pymongo import MongoClient

if not MONGO_URI:
    raise RuntimeError("Fant ikke MONGO_URI (hverken i secrets.toml eller

client = MongoClient(MONGO_URI, tls=True)
db = client[MONGO_DB]
coll = db[MONGO_COLL]

doc_count = coll.count_documents({})
print(f"Dokumenter i MongoDB → {MONGO_DB}.{MONGO_COLL}: {doc_count:,}")

print("\nTopp 3 dokumenter:")
for d in coll.find({}, {"_id": 1, "priceArea": 1, "productionGroup": 1, "
    print(d)
```

Dokumenter i MongoDB → elhub.production_2021_by_hour: 215,309

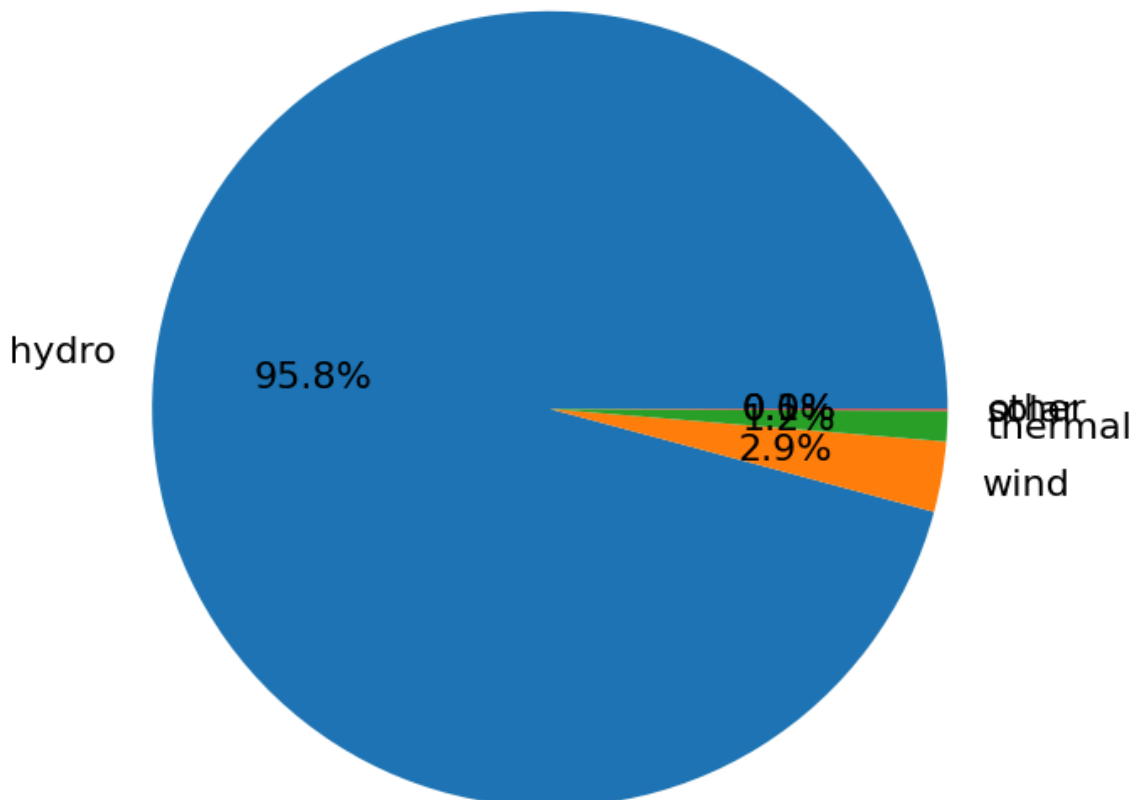
Topp 3 dokumenter:

```
{'_id': ObjectId('68f606ffa9c22d7386a5f1ff'), 'priceArea': 'N05', 'productionGroup': 'other', 'startTime': datetime.datetime(2020, 12, 31, 23, 0), 'quantityKwh': 0.0}
{'_id': ObjectId('68f606fda9c22d7386a5d003'), 'priceArea': 'N04', 'productionGroup': 'hydro', 'startTime': datetime.datetime(2020, 12, 31, 23, 0), 'quantityKwh': 3740830.0}
{'_id': ObjectId('68f606fca9c22d7386a5c7cc'), 'priceArea': 'N01', 'productionGroup': 'other', 'startTime': datetime.datetime(2020, 12, 31, 23, 0), 'quantityKwh': 0.0}
```

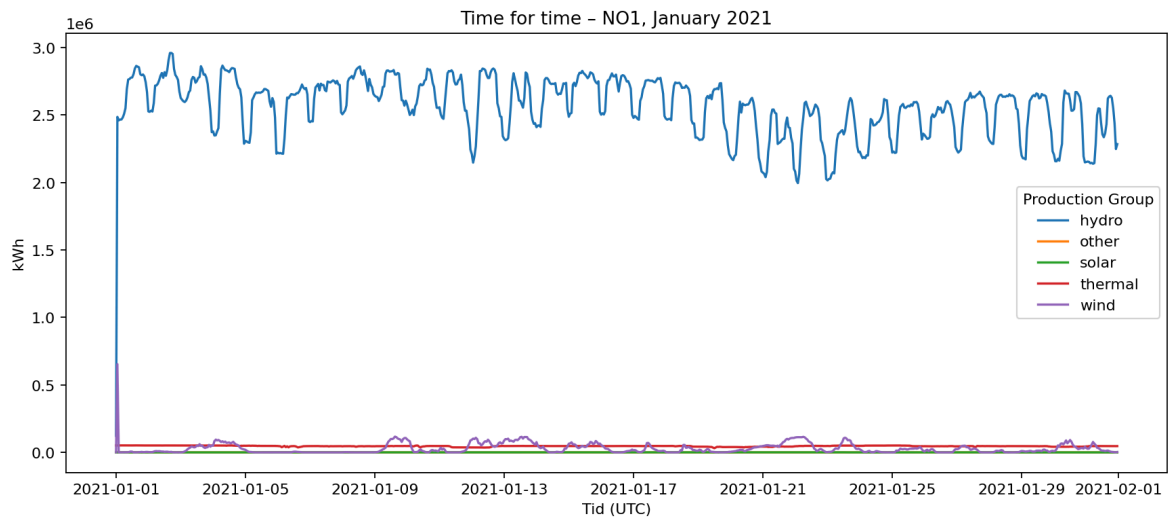
```
In [186... from IPython.display import Image, display
paths = [
    f"Ass_2/figs/pie_total_2021_{CHOSEN_AREA}.png",
    f"Ass_2/figs/line_2021_{CHOSEN_AREA}_01.png",
]
for p in paths:
    if os.path.exists(p):
        print("Viser:", p)
        display(Image(filename=p))
    else:
        print("Figur ikke funnet:", p)
```

Viser: Ass_2/figs/pie_total_2021_N01.png

Total produksjon 2021 – NO1



Viser: Ass_2/figs/line_2021_N01_01.png



Github log to show pull and push commands

```
In [187... import subprocess
import pandas as pd

# Run git log and capture output
result = subprocess.check_output(
    ["git", "log", "--pretty=format:%h|%an|%ad|%s", "--date=short"],
    text=True
)

# Split into lines and parse into DataFrame
rows = [line.split("|") for line in result.split("\n")]
df_log = pd.DataFrame(rows, columns=["Commit", "Author", "Date", "Message"])

df_log.head(20)
```

	Commit	Author	Date	Message
0	3f3487d	A H Sheikh	2025-10-22	Data Table: independent sparkline scaling; cle...
1	9cdf880	A H Sheikh	2025-10-20	Fix: 3_Plot.py repo-relativ CSV + fallback
2	238d061	A H Sheikh	2025-10-20	Streamlit side + MongoDB + utils – v2
3	76231b8	A H Sheikh	2025-10-20	Oppdatert utils_elhub, sider, Elhub page, Mong...
4	86da23b	A H Sheikh	2025-10-13	MongoDB test (TLS/certifi) OK; Spark⇌Cassandra...
5	b0ae33c	A H Sheikh	2025-10-13	Spark⇌Cassandra: Java 17, connector på classpa...
6	f6c5e7f	A H Sheikh	2025-10-08	Bekrefter Spark⇌Cassandra: la til 01_healthche...
7	765375d	A H Sheikh	2025-10-07	Opprettet ny branch 'utvikling_v2' for videre ...
8	9ed89f5	A H Sheikh	2025-09-30	IND320: oppdaterer Streamlit + utils; notebook...
9	0f8fbc9	A H Sheikh	2025-09-30	Added all plots in notebook, updated explanati...
10	3451f44	A H Sheikh	2025-09-29	La til requirements.txt for Streamlit Cloud
11	c96c5b5	A H Sheikh	2025-09-28	La til commits_github.txt i Innlevering og IPYNB
12	8d648f4	A H Sheikh	2025-09-28	Updated notebooks to use %run utils.ipynb inst...
13	48b3fa0	A H Sheikh	2025-09-28	Fix requirements.txt for Streamlit Cloud
14	e400a3e	A H Sheikh	2025-09-28	Fix utils.py to use relative path for CSV
15	0f65672	A H Sheikh	2025-09-28	Added dataset CSV and restored pages folder fo...
16	6df0632	A H Sheikh	2025-09-28	Moved streamlit_app.py and utils.py back to ro...
17	675022a	A H Sheikh	2025-09-28	Moved all notebooks and utils.ipynb into same ...
18	d8d2450	A H Sheikh	2025-09-28	Restored .py files for Streamlit app (misunder...
19	73fe7c2	A H Sheikh	2025-09-28	Added Jupyter notebooks (.ipynb) synced with ...